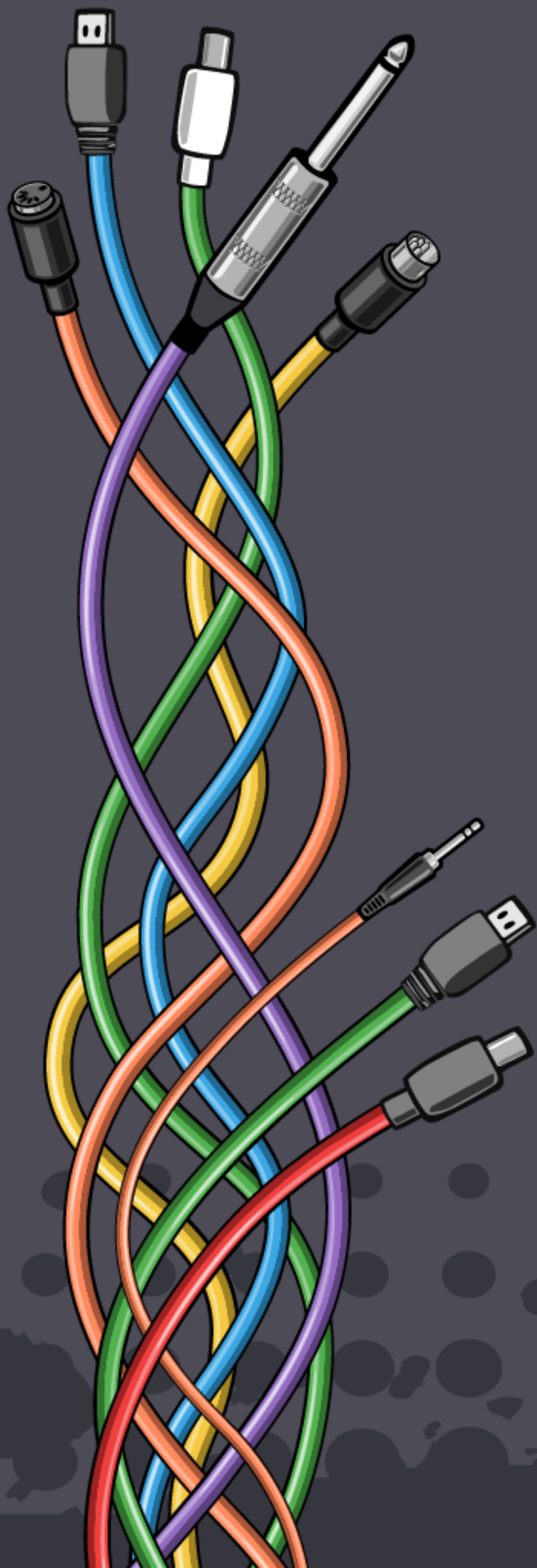




**WORKSHOP:**

# PROGRAMMING MUSIC AND SYNTHESIZERS ON-THE-FLY WITH PHARO

**DOMENICO CIPRIANI**

**ADC<sup>25</sup>**  
*Bristol*

# **(temporary) AGENDA**

**1. Foundations [ 45 minutes ]**

**2. Rhythm and Melodies [ 70 minutes ]**

☕ **coffee break! [ 10 minutes ]**

**3. Sound Design with Phausto [ 70 minutes ]**

**4. Performance [20 minutes ]**

**5. Visual Interface and Control [ 25 minutes ]**





# GOALS

- 1. Become friends with live-coding**
- 2. Meet and install Pharo, Coypu and Phausto**
- 3. Create rhythm and melodies with Coypu**
- 4. Create a small set of synthesisers with Phausto and perform with them.**
- 5. Design a basic User Interface with Bloc/CoypuIDE**

# DISCLAIMER/1

*Coypu and Phausto are still in an alpha stage, so we may encounter a few quirks (and maybe occasional crashes) .*

*Thanks for your patience and sense of adventure!*

*We're constantly improving them and can't wait to have your feedback. You're among the bold early explorers using tools that many others will only discover many years from now 🚀 .*

*We hope that exploring a new syntax and system of tools will be a smooth and inspiring experience.*



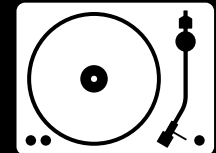
# DISCLAIMER/2

*Today we are learning useless stuff,*

*But that's how useful discoveries often begin.*

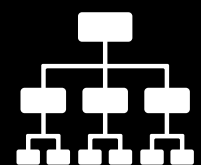
# About me

**Domenico Cipriani**  
**a.k.a. Lucretio (the Analogue Cops / Rage Therapy)**

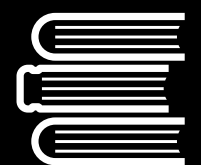


**Dj/live performer/live coder/electronic music producer**

- (Tresor, Berghain, Fabric, Rex, Concrete, Faust, LoveParade 2006, Dimensions Festival, Twisted Pepper... Barcelona, Frankfurt, Rome, Hanoi, San Francisco, Detroit, Belgrade, Tokyo, Osaka, Kiev...)
- More than 150 vinyl releases including records with Blawan, Objekt ...
- Performed at ICLC24 and ICLC25 (+workshop).



**Currently researching in computer music for the Evref team of Inria**



**Educator and Artist Partner for Soundwave, Italian distribution of Elektron machines**



# PART 1: FOUNDATIONS





# What is live coding?



- Live coding is about people interacting with the world, and each other, in real time, via code.
- Live coding is about making software live.
- Live coding is a performance practice that operates as an adventure and exploration, deliberately rejecting fixed definitions, remaining heterogeneous in nature.
- With no formal definition (or at least only one that includes the possibility of its own redefinition) also comes resistance to hierarchical control, live coding cannot really become owned by established practitioners or institutions

# TOPLAP

<https://blog.toplap.org>

(Temporary|Transnational|Terrestrial|Transdimensional) Organisation  
*for the*  
(Promotion|Proliferation|Permanence|Purity) of  
Live  
(Algorithm|Audio|Art|Artistic)  
Programming

informal organization formed in 2004 to bring together the communities formed around live coding environments.





# TOPLAP draft manifesto

<https://blog.toplap.org>

- Give us access to the performer's mind, to the whole human instrument.
- Obscurantism is dangerous. Show us your screens.
- Programs are instruments that can change themselves
- The program is to be transcended - Artificial language is the way.
- Code should be seen as well as heard, underlying algorithms viewed as well as their visual outcome.
- Live coding is not about tools. Algorithms are thoughts. Chainsaws are tools. That's why algorithms are sometimes harder to notice than chainsaws.





# What is an algorave?

- An **Algorave** is an event where people dance to music generated by algorithms, often performed through live coding
- Born in London in 2011 from the minds of **Alex McLean** and **Nick Collins**, it has since evolved into a worldwide community celebrating the art of coding sound and visuals.
- Code should be projected to the audience.
- Any **Algorithmic Music** is welcome when it is  
*“wholly or predominantly characterised by the emission of a succession of repetitive conditionals”*



# Thinking in many tongues

just to name a few

|                    |                                                                         |
|--------------------|-------------------------------------------------------------------------|
| <b>TidalCycles</b> | Haskell-based language for pattern-based algorithmic music.             |
| <b>ChuckK</b>      | Strongly timed language for on-the-fly sound synthesis and composition. |
| <b>Sonic Pi</b>    | Accessible Ruby environment for live-coded music and education.         |
| <b>ORCA</b>        | Esoteric, grid-based environment for composing and sequencing live.     |
| <b>Strudel</b>     | Web version of TidalCycles for browser-based live coding.               |
| <b>FoxDot</b>      | Python live coding interface for SuperCollider.                         |
| <b>Overtone</b>    | Clojure environment for synthesis, sampling, and collaborative music.   |
| <b>Mercury</b>     | Minimal language for web browser or MasMSP                              |
| <b>Extempore</b>   | Lisp-like language for real-time programming of sound and interaction.  |
| <b>Hydra</b>       | Browser-based system for coding visuals and video synthesis live.       |



# Pharo, Coypu, Phausto

## *PHARO*

Free, open-source, general-purpose language + cross-platform IDE

## *COYPU*

API and Domain Specific Language for programming music on-the-fly

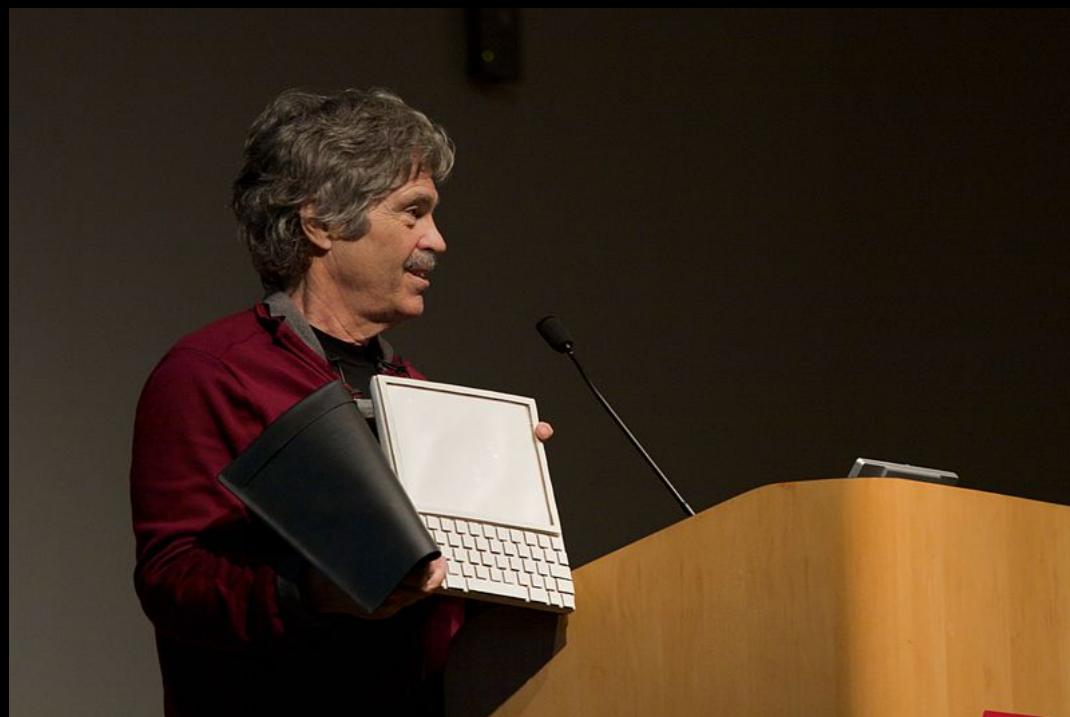
## *PHAUSTO*

Library and API for DSP programming that enables sound generation within Pharo



# Smalltalk

- A pure **Object-Oriented** Programming language developed at the *Learning Research Group* at Xerox PARC in the 1970s by Alan **Kay**, Dan **Ingalls**, Adele **Goldberg**, Ted **Kaeheler**.
- Designed for educational use following principles of *Constructionism*.
- Deeply influenced by **Simula**, developed by **Ole-Johan Dahl** and **Kristen Nygaard** in the 1960s at the Norwegian Computing Center in Oslo.
- Not just a language, also an IDE users can inspect and modify.
- Programs written in Smalltalk are compiled into byte code and interpreted by a virtual machine.
- Smalltalk has influenced *Objective C*, *Ruby*, *SuperCollider*.



# Pharo

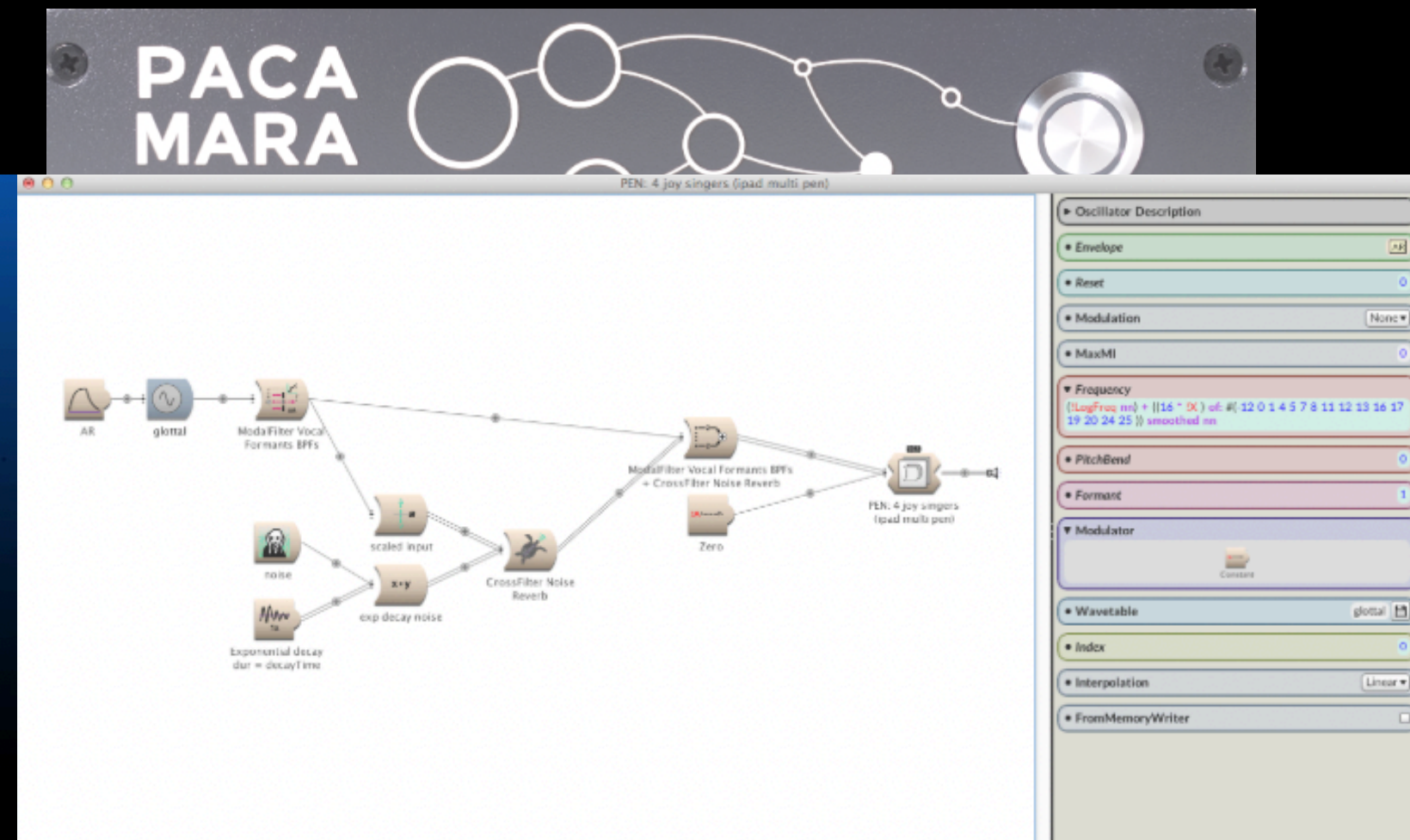


- A pure object-oriented, dynamically typed, and reflective language with a platform independent IDE
- Pharo was born as a fork of *Squeak Smalltalk*, led by **Stéphane Ducasse** and **Marcus Denker** at **INRIA Lille**.
- It is currently developed by INRIA, the Pharo Consortium, and a vibrant worldwide community
- Its syntax fits in a postcard.
- Integrated *Git* support and a framework for SUnit Tests.



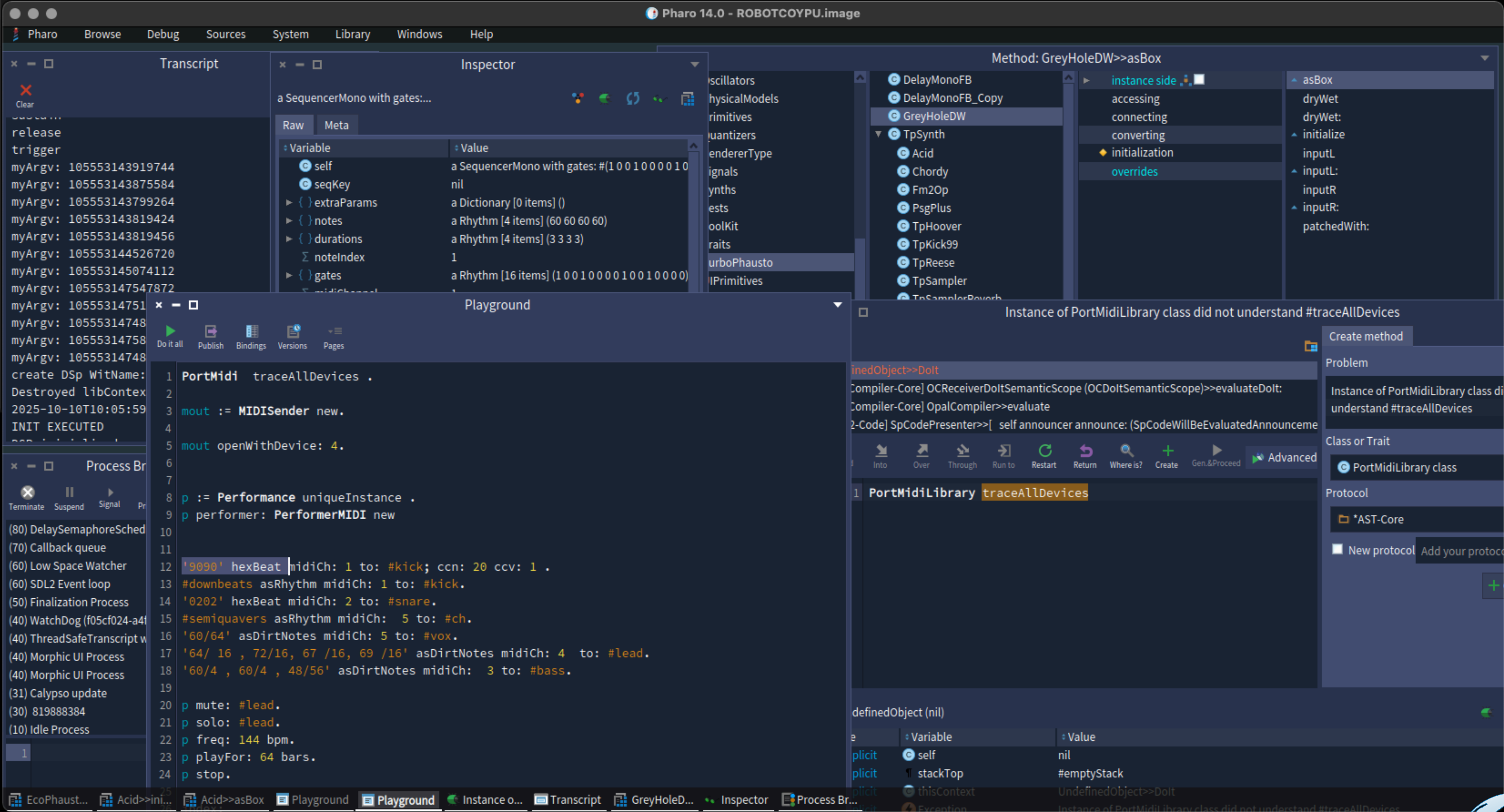
# Why Pharo?

- Born during the COVID-19 pandemic, **Coypu** grew from experiments in **Kyma**, whose closed-source limits inspired us to create an open-source OSC-based client.
- I chose Pharo for its modern design, extensive online resources, and approachable learning curve, and for its reflective, dynamic, and interactive programming model.





# The Pharo IDE



# Install Pharo

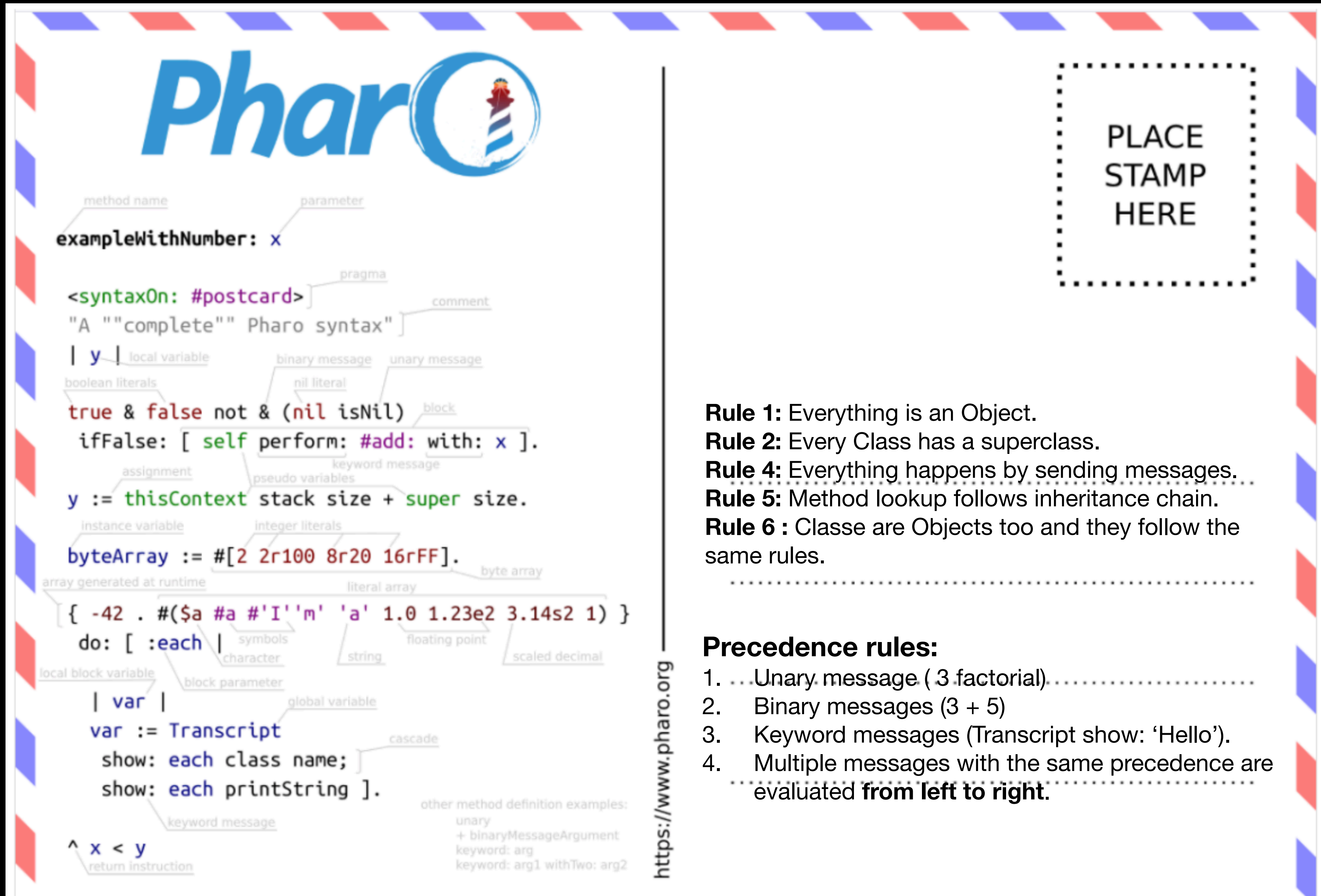
Download the Pharo Launcher to download a Pharo Image:

<https://pharo.org/download>

A Pharo image is an object space + Pharo Core Libraries + the virtual machine



# Pharo syntax on a postcard



PLACE  
STAMP  
HERE

## Rule 1: Everything is an Object.

## Rule 2: Every Class has a superclass.

## Rule 4: Everything happens by sending messages.

### Rule 5: Method lookup follows inheritance chain.

**Rule 6 :** Classe are Objects too and they follow the same rules.

## Precedence rules:

1. ... Unary message ( 3 factorial).....
2. Binary messages (3 + 5)
3. Keyword messages (Transcript show: 'Hello').
4. Multiple messages with the same precedence are evaluated **from left to right**.  
.....



# The Pharo IDE / shortcuts

|                             |                          |
|-----------------------------|--------------------------|
| <b><i>CMD/CTRL + OP</i></b> | <b>Open Playground</b>   |
| <b><i>CMD/CTRL + D</i></b>  | <b>Evaluate (Do)</b>     |
| <b><i>CMD/CTRL + P</i></b>  | <b>Print Object</b>      |
| <b><i>CMD/CTRL + I</i></b>  | <b>Inspect Object</b>    |
| <b><i>CMD/CTRL + B</i></b>  | <b>Browse Object</b>     |
| <b><i>CMD/CTRL + M</i></b>  | <b>List Implementors</b> |

# Installing Coypu and Phausto

Go to the *Coypu* GitHub repositories:

<https://github.com/lucretiomsp/Coypu>

Copy the *Metacello* script into your *Playground*.

```
Metacello new  
  baseline: 'Coypu';  
  repository: 'github://lucretiomsp/coypu:master';  
  load
```

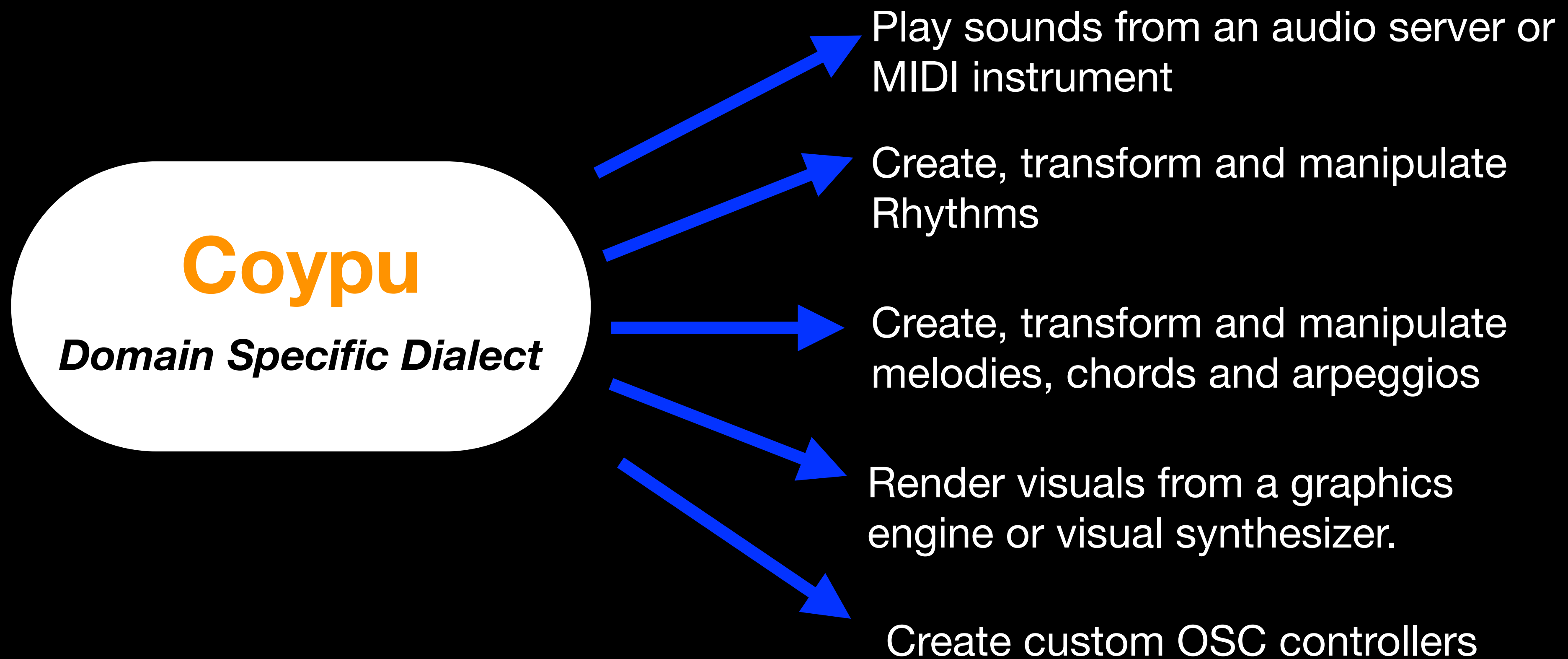
Select all the text and evaluate (**CMD/CTRL + D**)  
\*[*Coypu* already comes together with *Phausto*]

# PART 2: RHYTHM AND MELODIES



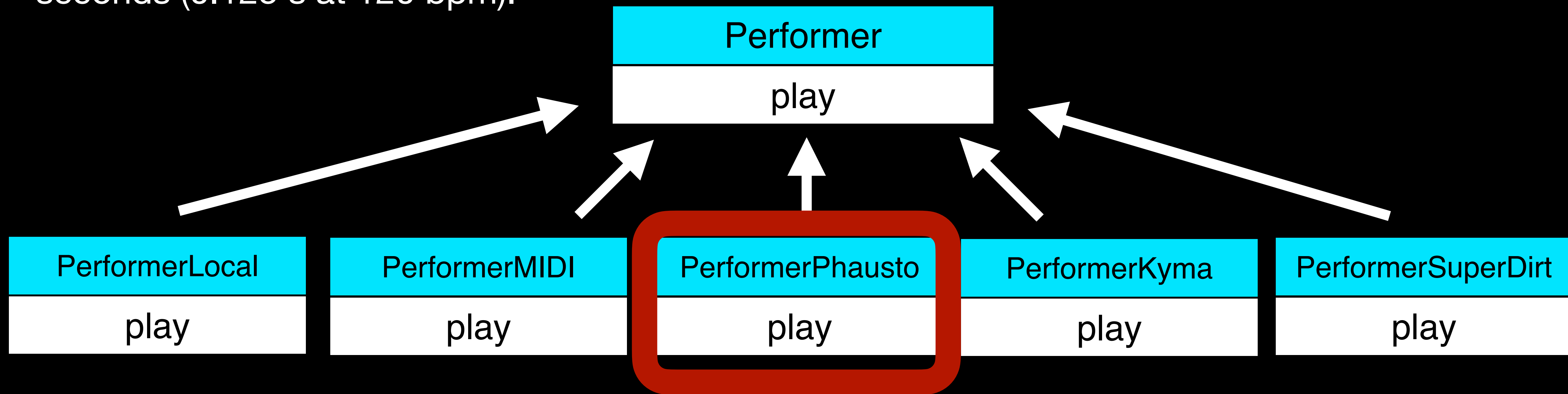


# Coypu's anatomy



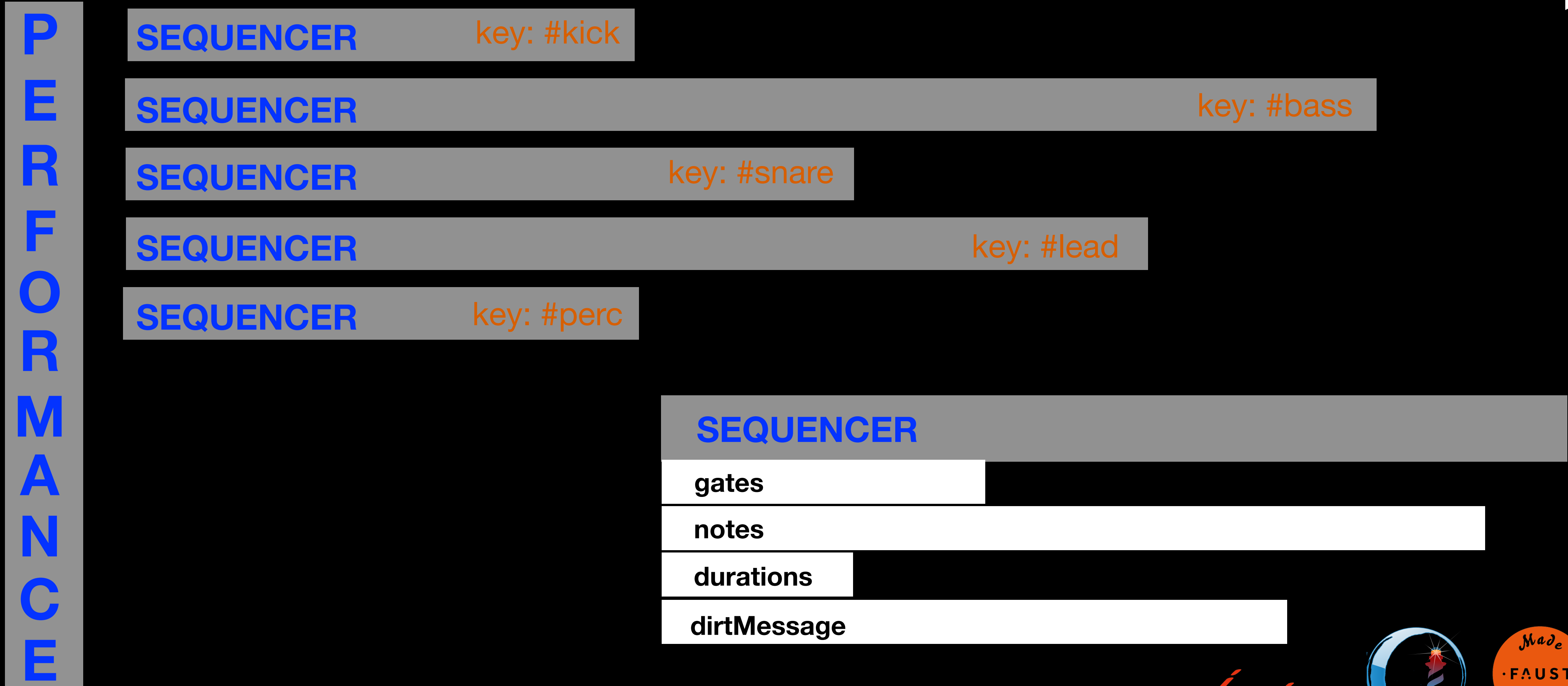
# Coypu's anatomy

- The **Performance** class is a *Singleton*.
- A **Performer** must be assigned to the Performance; the Performer selects the audio backend.
- The *play* method starts the Performance, and increments the transportStep every *freq* seconds (0.125 s at 120 bpm).



# Coypu's anatomy

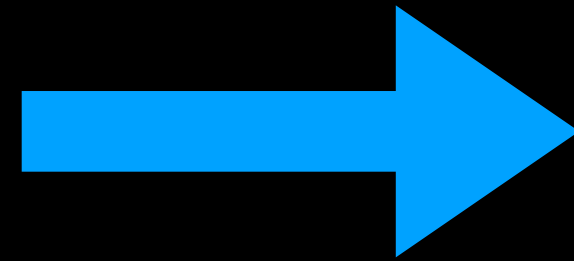
*transportStep* increments of 1 every **Performance** uniqueInstance freq (seconds)





# Principles

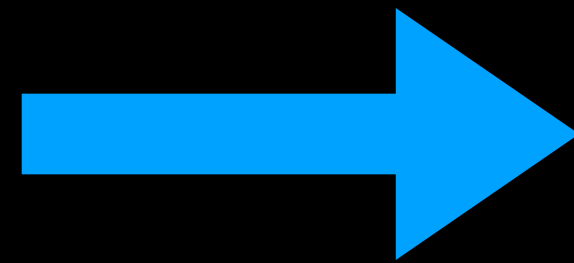
## Iconicity



Written code should resemble what we hear

16 upbeats

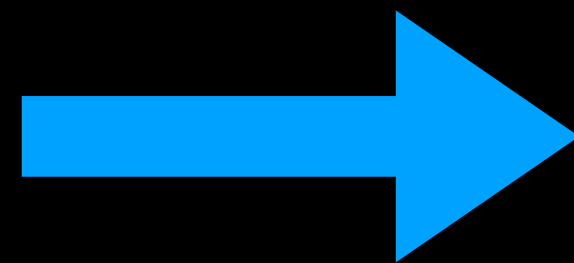
## Economy



The less we type, the better

#{60 63 67) + 16

## Polysemy



Many ways to do the same thing

16 downbeats.

#downbeats asRhythm.

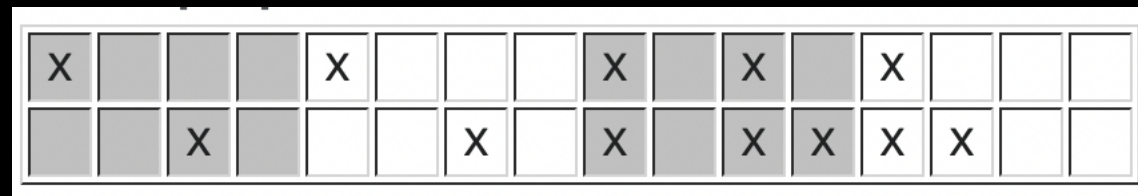
'60 , ~ , ~ , ~ , 60 , ~ , ~ , ~ , 60 , ~ , ~ , ~ , 60 , ~ , ~ , ~' asDirtNotes.

#{1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0) asSeq.

# Creating rhythms

- Sequencers are inspired by traditional hardware sequencers, where triggers (~noteOn) events are notated in **Time Unit Box System (TUBS)**.

TUBS



Traditional



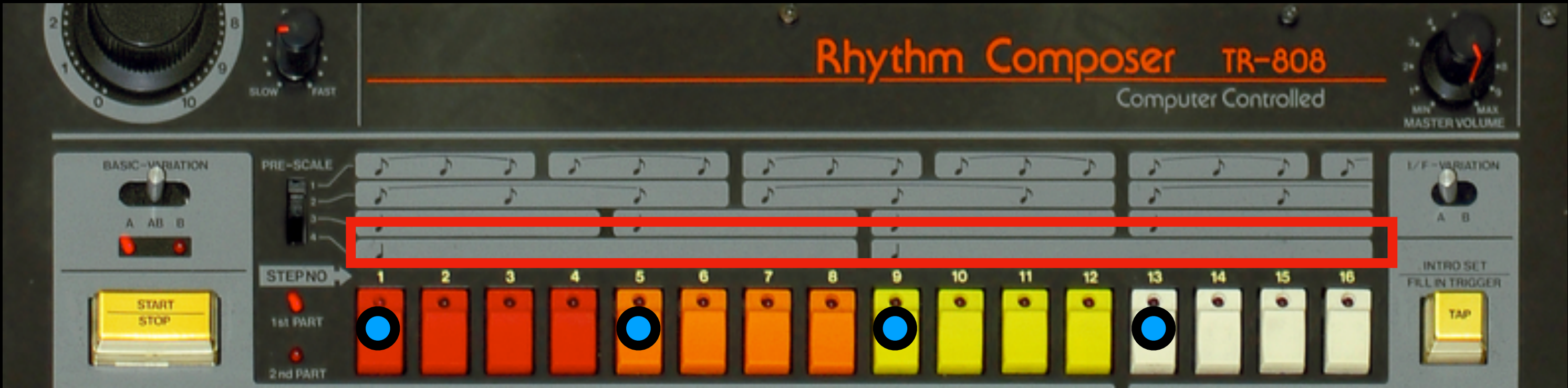
Coypu

#( 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 ) asSeq

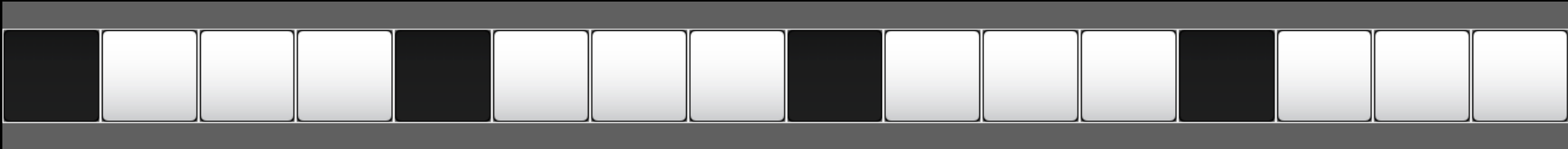
- Performance can be thought as a multitrack player for Sequencers.
- Sequencer are filled with parameters.
- **A Performer must be assigned to the Performance.**

```
1 p := Performance uniqueInstance. "assign the unique instance of the Performer class  
2 a variable"  
3 p performer: PerformerLocal new. "assign a local OSC performer to the Performance"  
4 p freq: 136 bpm. "change the performances speed"  
5 #( 1 0 0 0 1 1 0 0 1 0 ) asSeq notes: '32 37 38' to: #bass. "create a Sequencer and  
6 assign it to the performance"  
p playFor: 64 bars. "play the performance for 64 bars".  
p stop. "stop the performance"
```

# Creating rhythms/2



A rhythm can be represented as an array of 0s and 1s, where each 1 represents a trigger.



|       |                                                 |
|-------|-------------------------------------------------|
| PHARO | <code>#(1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0)</code> |
| COYPU | <code>#downbeats asRhythm</code>                |
| COYPU | <code>16 downbeats</code>                       |
| COYPU | <code>'8888' hexBeat</code>                     |



# Ponle la gasolina!



*(We will talk about Phausto after the break, now let's give it some gas! 🏁)*

```
1 TurboPhausto start.  
2 tp := TurboPhausto new.  
3 tp bpm: 167.  
4  
5 #junglekick' asRhythm to: #Kick.  
6 16 cumbiaClave to: #Marimba.a  
7 #(5 16) euclidean to: #Conga.  
8  
9 tp play.
```

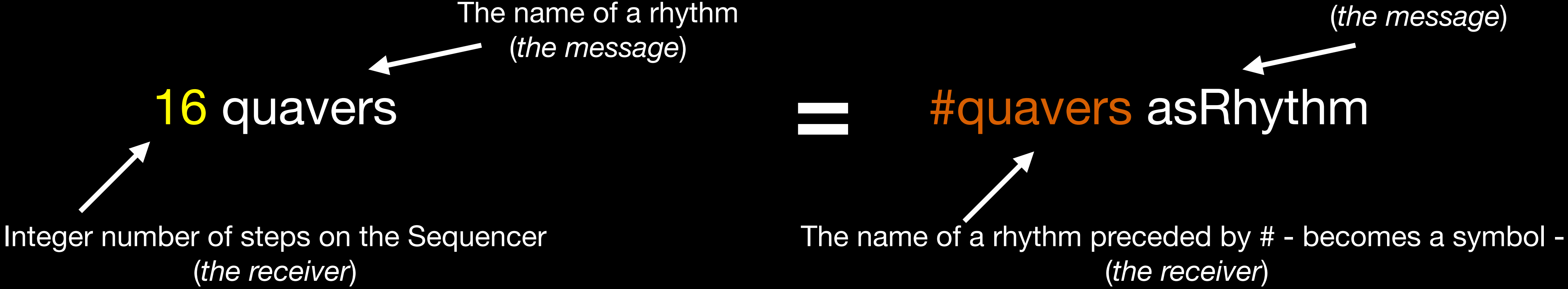
```
1 EcoPhausto start.  
2 ep := EcoPhausto new.  
3 ep bpm: 137.  
4  
5 '9090' hexBeat to: #Kick.  
6 16 quavers to: #Ch.  
7 16 randomTrigs to: #Bongo.  
8  
9 tp play.
```

# Turbo/EcoPhausto

- TurboPhausto is set of synthesisers and effects, *made with Phausto*, crafted for programming music on-the-fly with Coypu.
- Inspired by the SuperDirt audio engine for SuperCollider, it turns Pharo into a powerful environment for live-coded music and algoraves.
- Packed with 50 MB of algorave-ready royalty free audio samples, made by *Lucretio*, *The Analogue Cops* and the legendary dutch electro producer *Legowelt*.



# Creating rhythms/3



Rhythm list inspect



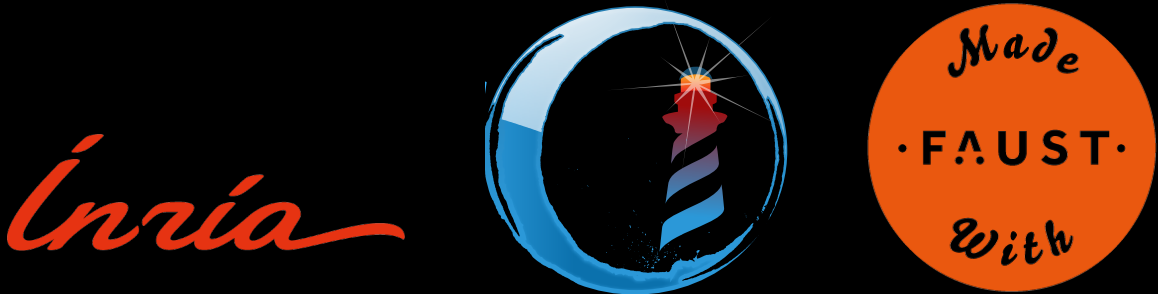
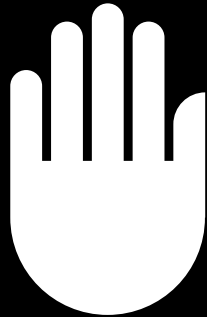
| Inspector                      |                                |
|--------------------------------|--------------------------------|
| a Dictionary [31 items] (#a... |                                |
| Items                          | Raw Meta                       |
| Key                            | Value                          |
| trueAksak                      | Balkan aksak rhythm pulses.    |
| wholes                         | Returns a rhythm sequence      |
| sikyi                          | create an array of self size o |
| claveSon                       | create an array of self size o |
| plena                          | create an array of self size o |
| rumba                          | create an array of self size o |
| cumbiaClave                    | cumbia clave rhythm            |
| jungleSnare                    | create an array of self size o |
| bossa                          | create an array of self size o |
| upbeats                        | creates an Array of size=self  |
| randomTrigs                    | generates an Array of rando    |
| tumbao                         | Habanera/tumbao rhythm pu      |
| shiko                          | create an array of self size o |
| bembe                          | In Toussaint's work on Eucli   |
| adowa                          | create an array of self size o |
| gahu                           | create an array of self size o |

Exceptions!

4 breves.

8 semibreves.

(Here, the integer receiver specifies the number of notes with that duration.)





# Creating rhythms/4



- #(1 0 0 1 0 0 0 1 0 0 1 0 1 0 0 0) asSeq
- #rumba asRhythm
- '9128' hexBeat



- #(1 0 1 0 1 0 1 0 1 0 1 1 1 1) asSeq
- 12 quavers, 4 semiquavers
- 'AAAF' hexBeat

# Creating rhythms/5

By default, every note in a sequencer is triggered using MIDI note number 60 (Middle C, or C4). The duration of each note is calculated as *sequencer size / number of trigs*, with a gate time of 0.8.

We can define arrays of note durations and/or gate times. These arrays don't need to have the same length as the number of trigs, which allows us to create polymetric sequencers—sequencers where each parameter can have its own rhythmic cycle.

For example:

**16** semiquavers notes: '38, 41 , 43' ; durations: '0.1 , 0.3' ; gateTime: '0.6, 0.5 , 0.4 , 0.1'

# Hexbeats

- 4 bits = 1 Nibble / 8 bits = 1 Byte.
- 1 Bar, 16 Steps (1/16th quantisation).
- If every step corresponds to a Bit of Information, in a Bar we find 16 Bits, (2 Bytes).
- In every Bar there are 4 Beats, so in each Beat there are 4 Bits, i.e. 1 Nibble.
- Every Hexadecimal symbol represents a Nibble (4 steps

|   |   |   |   |   |             |
|---|---|---|---|---|-------------|
| 0 | 0 | 0 | 0 | 0 |             |
| 1 | 0 | 0 | 0 | 1 |             |
| 2 | 0 | 0 | 1 | 0 | Upbeats     |
| 3 | 0 | 0 | 1 | 1 |             |
| 4 | 0 | 1 | 0 | 0 |             |
| 5 | 0 | 1 | 0 | 1 |             |
| 6 | 0 | 1 | 1 | 0 |             |
| 7 | 0 | 1 | 1 | 1 |             |
| 8 | 1 | 0 | 0 | 0 | Downbeats   |
| 9 | 1 | 0 | 0 | 1 |             |
| A | 1 | 0 | 1 | 0 | Quavers     |
| B | 1 | 0 | 1 | 1 |             |
| C | 1 | 1 | 0 | 0 |             |
| D | 1 | 1 | 0 | 1 |             |
| E | 1 | 1 | 1 | 0 |             |
| F | 1 | 1 | 1 | 1 | Semiquavers |

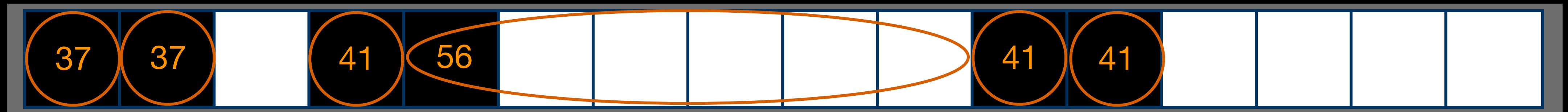


# Random rhythms and melodies

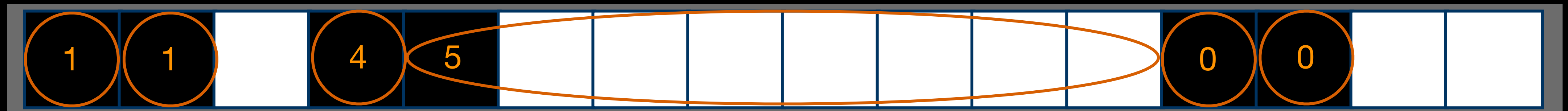
```
1 16 randomRhythm.  
2 32 randomTrigs.  
3 64 randomTrigWithProbability: 70.  
4 24 plena randomNotesIn: Scale istrian + 46.  
5 (32 semiquavers randomNotesFrom: #(48 60 63 56)).  
6 (64 semiquavers randomWalksOn: Scale sakura octaves: 3 root: 48).  
7  
8 gen := MarkovJazzMelodyGenerator new.  
9 melody := gen generateMelodyFromScaleDegrees: Scale bebop root: 60 startDegree: 0 length: 32.
```

# The 'musky' notation

A Tidal Cycles 'string notation' to create Sequencers with different notes or different samples



- '37 \* 2, ~, 41, 56 / 8, 41 \* 2, ~ \* 4' asDirtNotes



- '1 \* 2, ~, 4, 5 / 8, 0 \* 2, ~ \* 2' asDirtIndex

# Scales

- Sending a scale name to the **Scale** class returns an array of intervals for that scale.
- You can add a root note using **+** to shift all intervals.
- Or get a full array of notes across octaves by sending:

**Scale** root: <rootNote> octave: <n>

This gives all the notes starting from the root across **n** octaves.

- **Scale** sakura ^ #(0 1 5 7 8).
- **Scale** sakura + 60 ^ #(60 61 65 67 68).
- **Scale** sakura root: 36 octave: 2 ^ # (36 37 41 43 44 48 49 53 55 56 60 61 65 67 68)



# Sequencer operations

Combine Sequencers with ,

`#junglekick' asRhythm / #downbeats asRhythm / 'AAAF' hexBeat`

Repeat Sequencers with \*

`'AAAF' hexBeat * 3`

Transpose Sequencers with +

`#semiquavers hexBeat + 7`

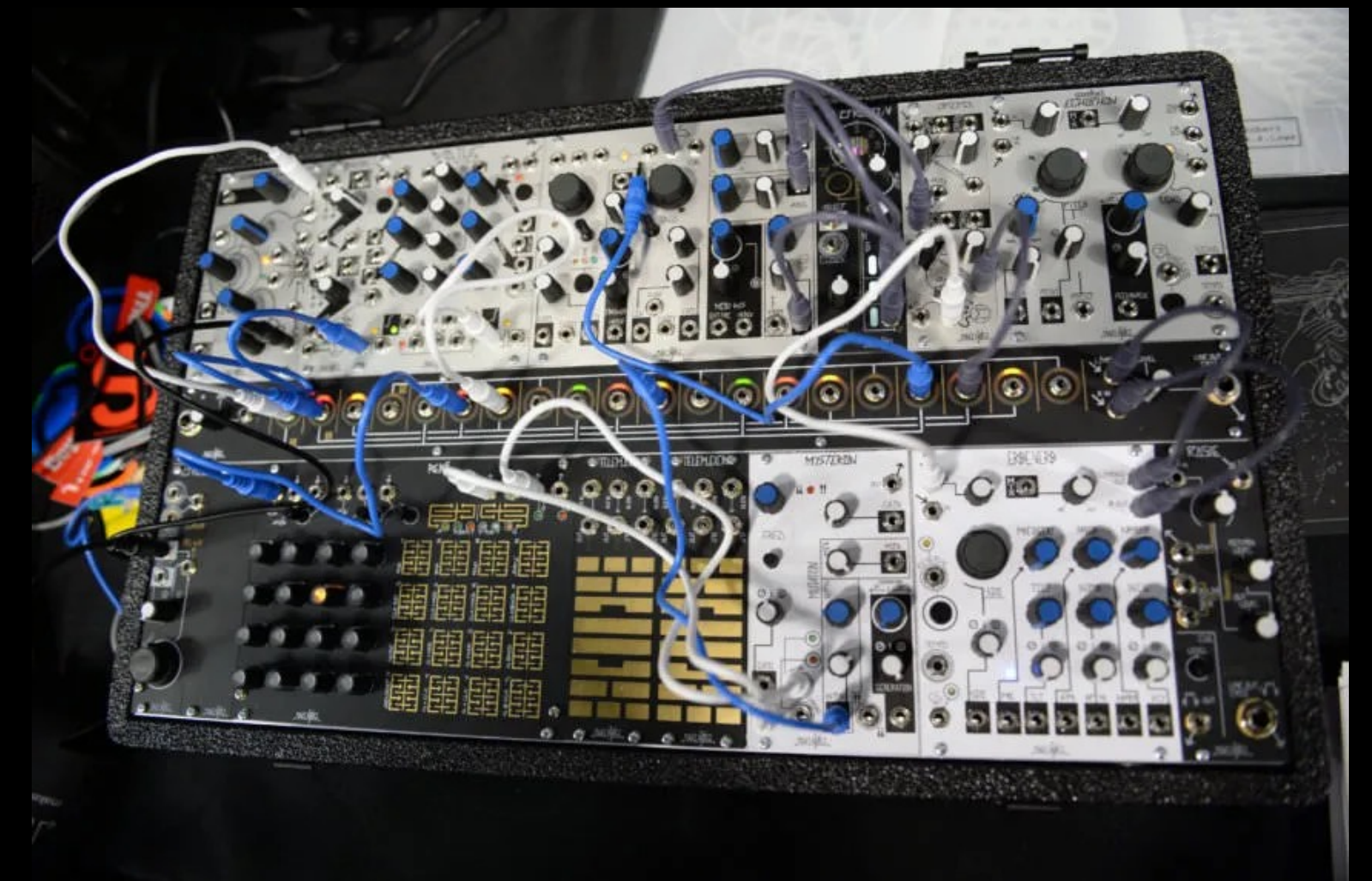
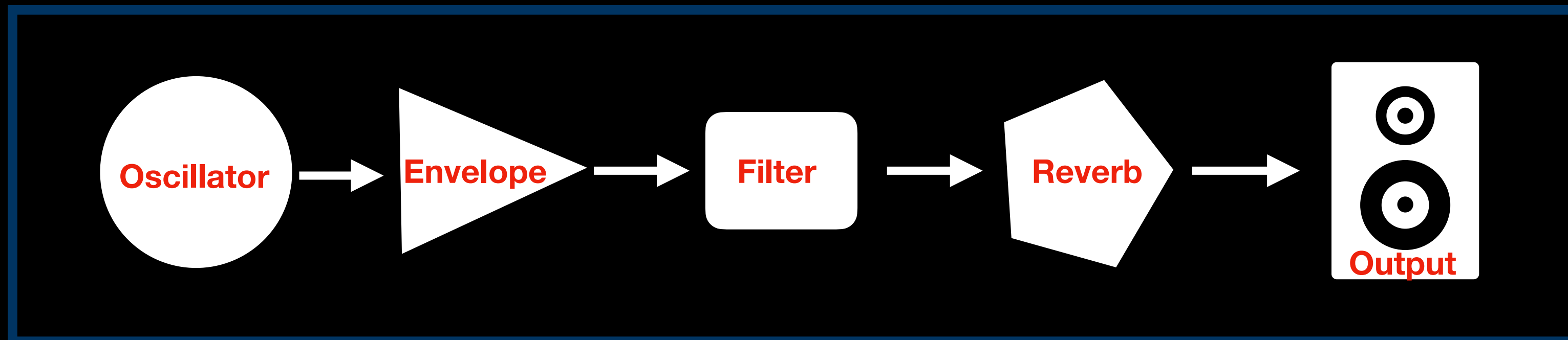
# PART 3: SOUND DESIGN WITH PHAUSTO





# Start your engine

- Phausto is a library and API for sound generation and DSP programming within the Pharo IDE, powered by an embedded Faust compiler.
- Phausto brings a modular-synth-inspired approach to designing synths and effects.
- Unit Generators are connected by setting their members value or using the **ChuckK** operator => .





# Time for a pit stop



```
1 TurboPhausto start.  
2 tp := TurboPhausto new.  
3 tp bpm: 167.  
4  
5 #junglekick' asRhythm to: #Kick.  
6 16 cumbiaClave to: #Marimba.  
7 #(5 16) euclidean to: #Conga.  
8  
9 tp play.
```

```
1 EcoPhausto start.  
2 ep := EcoPhausto new.  
3 ep bpm: 137.  
4  
5 '9090' hexBeat to: #Kick.  
6 16 quavers to: #Ch.  
7 16 randomTrigs to: #Bongo.  
8  
9 tp play.
```

- The **start** method loads all instruments and effect and create a **DSP** that is assign to a class variable named **tpDsp**.
- It also initialise a new **Performance** (also stored in a class variable) and connect it to the **DSP**.

# Let's build a synth

```
1 oscillator := Pulse0sc new label: 'MySynth'.
2 env := ADSREnv new label: 'MySynth'.
3 filter := Korg35LPF new label: 'MySynth'.
4 effect := GreyHoleDW new label: 'MySynth'.
5
6 synth := oscillator => env => filter => effect.
7 dsp := synth stereo asDsp.
8
9 dsp traceAllParams.
10 dsp init.
11 dsp start.
12 dsp playNote: 60 prefix: 'MySynth' dur: 1.
```

# Drive Phausto with Coypu

```
1  p := Performance uniqueInstance .  
2  p forDsp: dsp.  
3  
4  '38, 56, 65, ~ , 42 , 76 , 81 , 45' asDirtNotes to: #MySynth.  
5  
6  p freq: 129 bpm.  
7  p playFor: 16 bars.  
8  
9  
10  
11  
12
```



# Sum of synths

```
1 dsp := (Kick new + Clap new + CombString new + Hat new + PopFilterDrum new + SawTrombone new + DubDub new) stereo asDsp.
2
3 dsp init.
4 dsp start.
5
6 p := Performance uniqueInstance .
7 p forDsp: dsp.
8
9 '8080' hexBeat to: #Kick.
10 '38, 56, 65, ~ , 42 , 76 , 81 , 45' asDirtNotes to: #DubDub.
11 #rumba asRhythm to: #Hat.
12 32 randomTrigs to: #Clap.
13 '60/32 , 63/16 ' asDirtNotes to: #SawTrombone.
14 #( 7 16 euclidean) to: #PopFilterDrum.
15
16 p playFor: 16 bars.
```

# MasterLu

Our First Synthesiser (8/15)

Do it all

Publish

Bindings

Versions

Pages

```
1 "lets create our first subtractive synthesiser"
2
3 "first we need at least one oscillator"
4 oscillator := PulseOsc new.
5 "then we need an envelope, in this case a linear Attack-Decay-Sustain-Release envelope"
6 envelope := ADSREnv new.
7 "also we create a filter, in this case a Moog Vcf lowpass filter"
8 filter := MoogVcf new.
9
10 "to construct our synthesiser we chuck the Oscillator into the Envelope (hence the Oscillator is multiplied by the envelope - and we chuck this composite Unit Generator into the filter, this means that the composite Unit Generator is the input of the filter".
11
12 synth := oscillator => envelope => filter.
13 "binary operators have left-to-right precedence in Pharo, so we just place the Unit Generators in chain"
14
15 "now we create a stereo DSP"
16 dsp := synth stereo asDsp.
17 "we initialize the DSP"
18 dsp init.
19 "we start it"
20 dsp start.
21
22 "to hear the sound we need to trigger the envelope, so we must open the UI and press the button"
23 dsp displayUI .
24
25 "now have fun tweaking the sliders, try to understand what happens to the sound".
26
27 "and then we stop the DSP before jumping to the next lesson"
28 dsp stop.
29 MasterLu next.
```

Line: 29:15

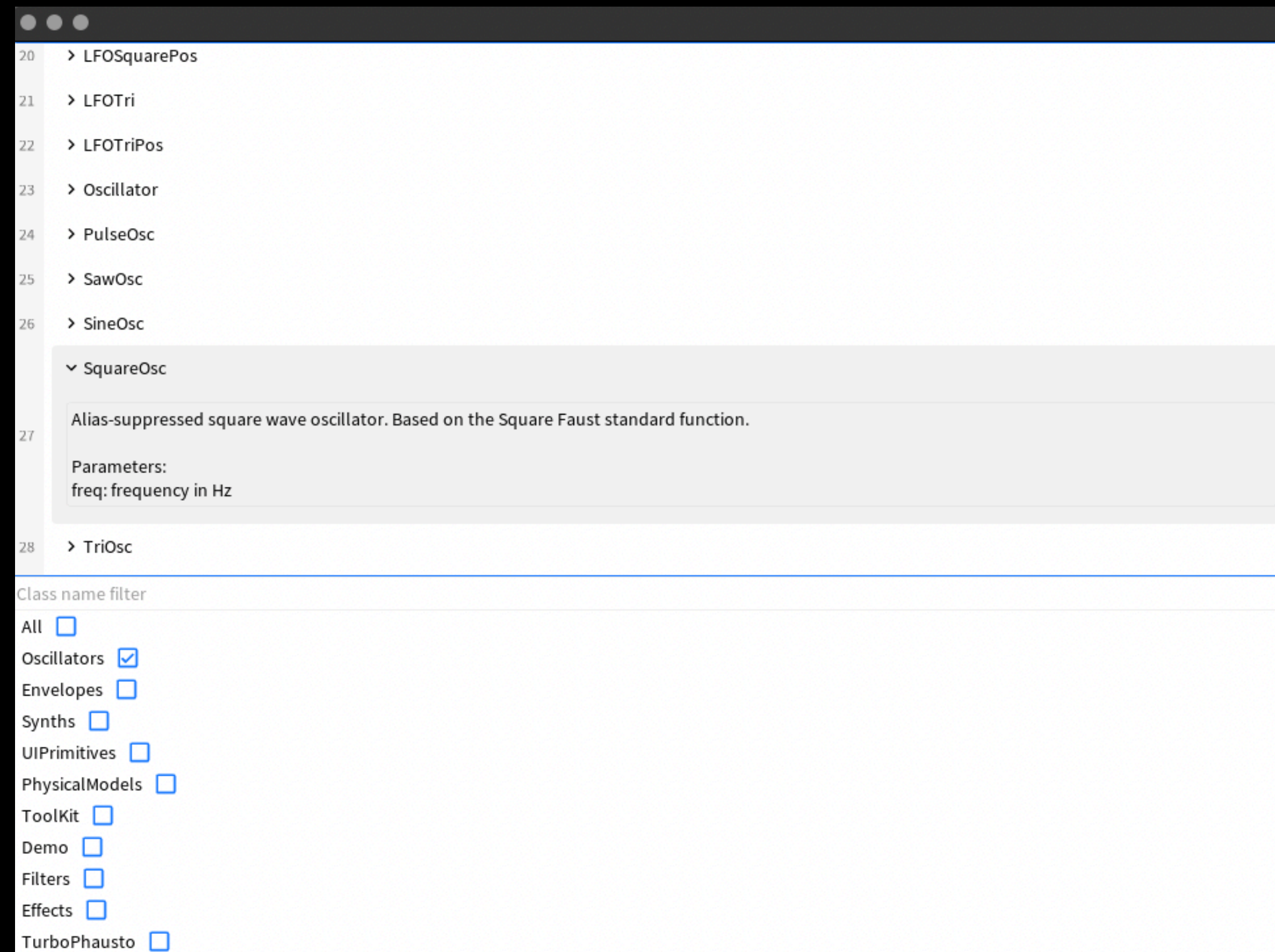
```
Metacello new
  baseline: 'MasterLu';
  repository: 'github://lucretiomsp/MasterLu:main';
  load
```

MasterLu go.  
MasterLu next.

***MasterLu** is your electronic music teacher. 15 lesson everything you need to know*



# ClassReference



PhaustoClassReference new openInSpace.



# Build your CustomPhausto



**Exercise!**

- Study the *EcoPhausto* class and design a subclass called *CustomPhausto*.
- Your design should include multiple sound sources, at least two *TpSamplers* and two *Synths*, drawn from Phausto's existing set or custom ones you define.
- When your *CustomPhausto* is complete, we'll be ready to perform with it and *Coypu*.

# PART 4: PERFORMANCE



# Tuning Phausto for Coypu

|   |                                                                                                |
|---|------------------------------------------------------------------------------------------------|
| 1 | <code>tablas := TpSampler new label: 'tablas' ; pathToFolder: '/samples/tabla'.</code>         |
| 2 | <code>viola := TpSampler new label: 'viola' ; pathToFile: '/samples/strings/viola.wav'.</code> |
| 3 | <code>pulsar := (PulseOsc new label: 'pulsar') =&gt; ADSR new label: 'pulsar';</code>          |
| 4 | <code>synths := (table + viola + pulsar) stereo asDsp.</code>                                  |
| 5 |                                                                                                |
| 6 |                                                                                                |

**⚠ Labels should begin with minor case letters!**



# Nibbling things up

```
p := Performance uniqueInstance .  
"Any Performer can be used"
```

```
p freq: 129 bpm.  
p playFor: 128 bars.
```

```
p mute: #kick.  
p mute: #(#clap #bass).
```

```
p solo: #bongo.  
p unsolo: #bongo.
```

```
p backup.
```

```
p muteAll.  
'AAAAAAF' hexBeat to: #snare.  
p restore
```





# Nibbling things up/2

```
EcoPhausto start.  
ep := EcoPhausto new.  
  
ep playFor: 128 bars.  
  
downbeats #asRhythm to: #kick.  
rumba #asRhythm to: #kick.  
#kick swapWith: #snare.  
ep solo: #bongo.  
  
ep solo: #bongo.  
ep unsolo: #bongo.  
  
ep backup.  
  
#(5 16 euclidean) to: #bongo.  
#(5 16 euclidean) offset: 2)to: #conga. "Also try offsetLeft or offsetRight!"  
  
ep restore.
```



# Extemporaneous Music

- We invite you to perform without synchronisation. Follow the sounds coming next to you. Let collective time arise from listening.
- **Extemporaneous Music** explores sound as a spontaneous events, echoing the ideas of sonic pointillism where each sound exists in its own space and time.
- A performance created *ex tempore*, emerges from real-time interaction between performers and machines.
- This approach proposes an alternative musical form: collective, algorithmic, improvised, and continuously unfolding in the moment and alive in the moment

# PART 5: VISUAL INTERFACE





# Bloc

- **Bloc** is a powerful low-level graphical framework for Pharo, created by **Alain Plantec** and enhanced by the *Feenk* team for *GToolkit*. These combined efforts are now merging back into Pharo, significantly advancing its graphical capabilities.
- **Bloc** is poised to become the primary graphical framework for Pharo, gradually replacing the well-established but aging Morphic framework.
- **Bloc** is under active development, so features and behavior may change as it matures.



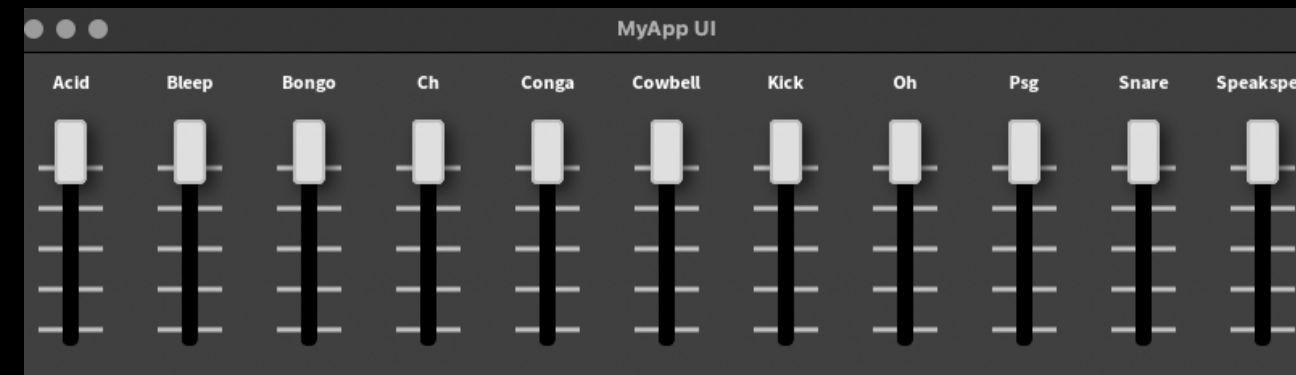
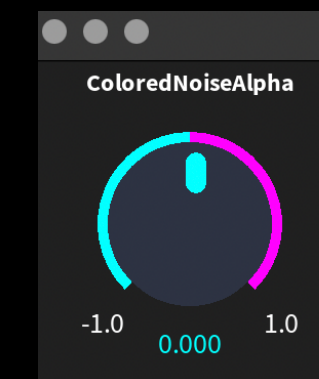
# CoypuIDE

- CoypuIDE is a set of widgets for the Coypu and Phausto music and DSP libraries, implemented in Bloc and Toplo.
- It comes together with the MasterLu package or can be installed separately with a script:

```
Metacello new  
  baseline: 'CoypuIDE';  
  repository: 'github://pharo-graphics/CoypuIDE:master';  
  load
```

# CoypuIDE/2

```
1 dsp := Clap new stereo asDsp.  
2 dsp init; start.  
3 "Display the full UI"  
4 dsp displayUI.  
5  
6 dsp := ColoredNoise new stereo asDsp.  
7 dsp init; start.  
8 dsp traceAllParams.  
9 widget := dsp widgetForParameter: 'ColoredAlphaNoise'.  
10 Widget openInSpace.  
11  
12 EcoPhausto start.  
13 ep := EcoPhausto new.  
14 ep displayUIForLevels.
```



# Join the community



## Pharonauts on Discord